iOS Patch Diffing

Análise de Patches de Segurança iOS

Bruno Naves · Navees





```
$ ipsw diff --patch --analyze CVE-2026-20667
```

Agenda

01

Whoami

Speaker background

02

Arquivos iOS

IPSW, dyld cache, kernel cache

03

Patch Diffing

Conceito, fluxo e importância

04

Tools

ipsw, Ghidriff, Meld e mais

05

CVE-2026-20667

Análise completa

01



Whoami

Speaker background
e credenciais

Bruno Naves

Role

Offensive Security Engineer

Fundador

Bsides BSB

Focus

Pesquisador iOS / Mobile

Admin

Apt Zé da Manga

Certs

OSCE3 · OSWE · OSEP · OSED · OSCP · OSWP · CRTO · CRTP · iASE · SCMP|A

Membro

Mobile Breakers

```
$ whoami → @navees | iOS Security Research
```

02



Arquivos iOS

IPSW · dyld shared cache
· Kernel Cache

Arquivos iOS

dyld Shared Cache

`.dylib cache`

Mecanismo de otimização nos sistemas Apple (macOS, iOS) que agrupa bibliotecas dinâmicas (.dylib) comumente usadas em um único arquivo de cache carregado na memória durante a inicialização.

Kernel Cache

`kernelcache`

Arquivo contendo o núcleo do sistema operacional (Kernel) e extensões essenciais, pré-carregados para acelerar a inicialização do iPhone.

IPSW

`.ipsw firmware`

Arquivo de firmware da Apple que contém todos os componentes do sistema operacional, usado para atualização, restauração e análise do sistema.

03

O que é Patch Diffing?

Comparação binária entre
versões vulnerável e corrigida

O que é Patch Diffing?

Patch diffing é o processo de comparar duas versões de um binário — uma vulnerável e outra corrigida — para determinar o que mudou. No iOS, essa técnica é amplamente utilizada para entender as correções de segurança da Apple, descobrir a causa raiz de CVEs e identificar novas superfícies de ataque.



Binário v1 (vulnerável) vs. Binário v2 (corrigido) → comparar funções, dados e estruturas.



Ajuda a identificar quais funções foram adicionadas, removidas ou modificadas.



No iOS, envolve trabalhar com arquivos IPSW, kernelcache e dyld_shared_cache.



Algumas mudanças também podem estar em código open source (XNU, WebKit, etc.).

Por que Patch Diffing é importante?

01 Pesquisa de Vulnerabilidades

Verificar se o patch da Apple realmente corrigiu o problema ou se foi apenas uma correção superficial.

02 Criação de N-day Exploit

Depois que a causa raiz é identificada, ela pode ser usada para criar um exploit N-day a partir de vulnerabilidades já divulgadas.

03 Threat Hunting & Análise de Malware

Comparar similaridades entre atores de ameaça e seus malwares, ou relacionar novos malwares com famílias já conhecidas.

Patch Diffing — Pontos de Atenção



Mudanças não-security

Às vezes pode ser uma mudança nas propriedades do binário, como entitlements — nem tudo é vulnerabilidade de segurança.



Mudanças funcionais vs. security

Nem todas as mudanças em um binário vulnerável são causadas por uma vulnerabilidade. Algumas são apenas mudanças funcionais ou de performance.



Faca de dois gumes

Ajuda pesquisadores a fortalecer a segurança, mas também fornece a atacantes insights sobre vulnerabilidades recém-corrigidas (N-day).

Fluxo de Patch Diffing

1. Identificar CVE nas notas de lançamento da Apple; buscar write-ups públicos.
2. Baixar dois IPSWs: versão corrigida + versão anterior (N-1).
3. Determinar arquivos atualizados (ex: ipsw diff).
4. Mapear binários para o CVE — o diff ajuda a reduzir escopo.
5. Extrair binários: kernelcache, dyld_shared_cache, libs, daemons...
6. Mapear símbolos (se possível). A ferramenta ipsw pode ajudar.
7. Comparar binários com: meld, ghidriff, Diaphora, BinDiff ou manualmente.
8. Encontrar a causa raiz — localizar a mudança exata no código.

Identifying CVE — iOS Security Releases

About the security content of iOS 26.3 and iPadOS 26.3

This document describes the security content of iOS 26.3 and iPadOS 26.3.

About Apple security updates

For our customers' protection, Apple doesn't disclose, discuss, or confirm security issues until an investigation has occurred and patches or releases are available. Recent releases are listed on the Apple security releases page.

Apple security documents reference vulnerabilities by CVE-ID when possible.

For more information about security, see the Apple Product Security page.

iOS 26.3 and iPadOS 26.3

Released February 11, 2026

Accessibility

Available for: iPhone 11 and later, iPad Pro 12.9-inch 3rd generation and later, iPad Pro 11-inch 1st generation and later, iPad Air 3rd generation and later, iPad 8th generation and later, and iPad mini 5th generation and later

Impact: An attacker with physical access to a locked device may be able to view sensitive user information

Description: An inconsistent user interface issue was addressed with improved state management.

CVE-2026-20645: Loh Boon Keat

04



Tools

ipsw · Ghidriff · Binary Ninja
Meld · Diaphora · BinDiff

Ferramentas para Patch Diffing

ipsw — CLI
Download e diff de Apple Firmware

Ghidriff — Python
Binary diffing baseado no Ghidra

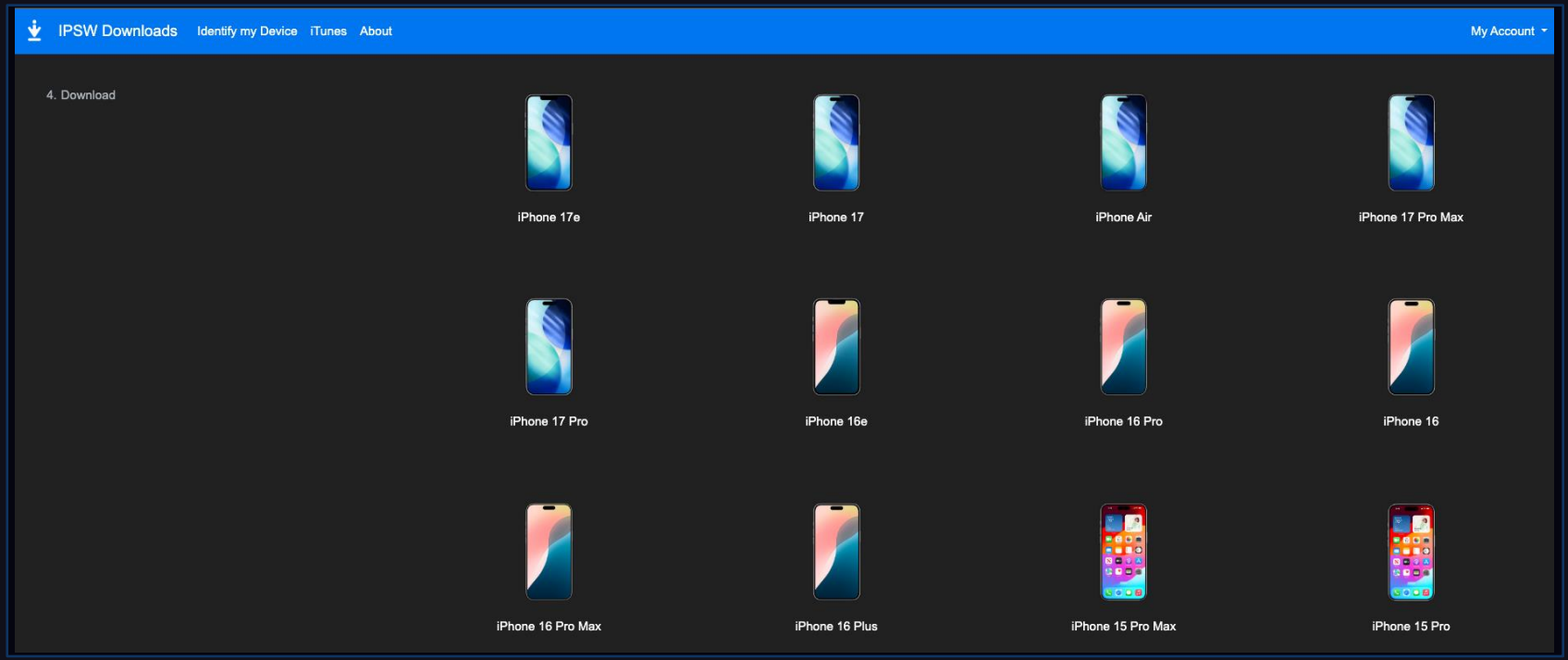
Ghidra / BinNinja / IDA — RE Tool
Eng. reversa de Mach-O e dyld_shared_cache

Meld — GUI
Ferramenta gráfica de diff para comparações visuais

Diaphora / BinDiff — IDA Plugin
Engine avançada para binary diffing

IPSW

IPSW — Download de Firmware Apple



IPSW — iOS/macOS Research Swiss Army Knife



ipsw

iOS/macOS Research Swiss Army Knife

Go passing | downloads 301k | release v3.1.665 | license mit

What is `ipsw` 🤔

ipsw is a comprehensive command-line research framework for iOS and macOS. It provides an extensive toolkit for security researchers, reverse engineers, jailbreak developers, and iOS enthusiasts to download, parse, and analyze Apple firmware and interact with iOS devices.

Core Capabilities

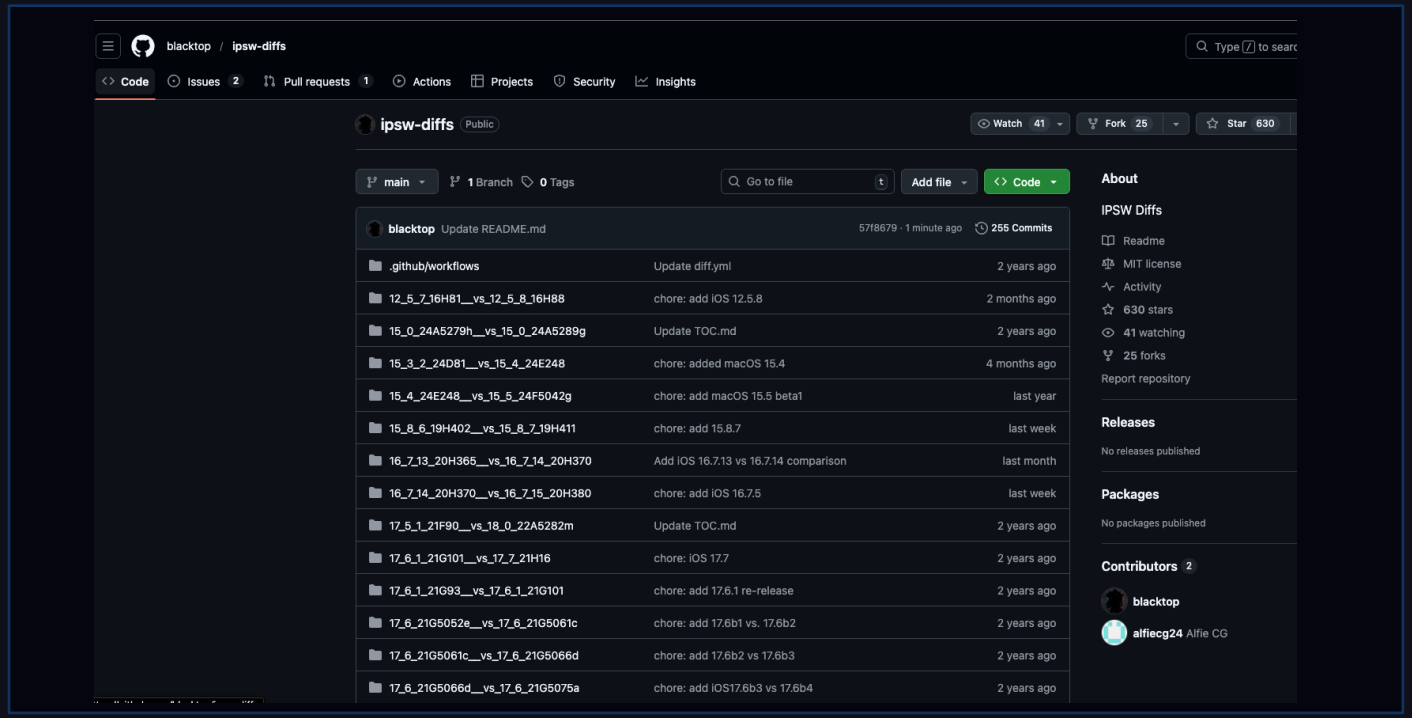
- 📱 **IPSW/OTA Analysis** - Download, extract, and analyze iOS firmware files
- 🔍 **Binary Analysis** - Advanced Mach-O parsing with ARM disassembly and AI assistance
- 🧠 **dyld_shared_cache** - Complete shared cache analysis with ObjC/Swift class dumping
- 🔧 **Kernel Analysis** - Kernelcache parsing, syscall extraction, and symbolication
- 📲 **Device Interaction** - Comprehensive iOS device management and debugging
- 🔐 **Firmware Research** - IMG4, iBoot, SEP, and co-processor firmware analysis
- 🏪 **App Store Connect** - Full API integration for app and certificate management
- 🛠️ **Developer Tools** - SSH, Frida, debugging, and reverse engineering utilities

Usando IPSW para Diffing

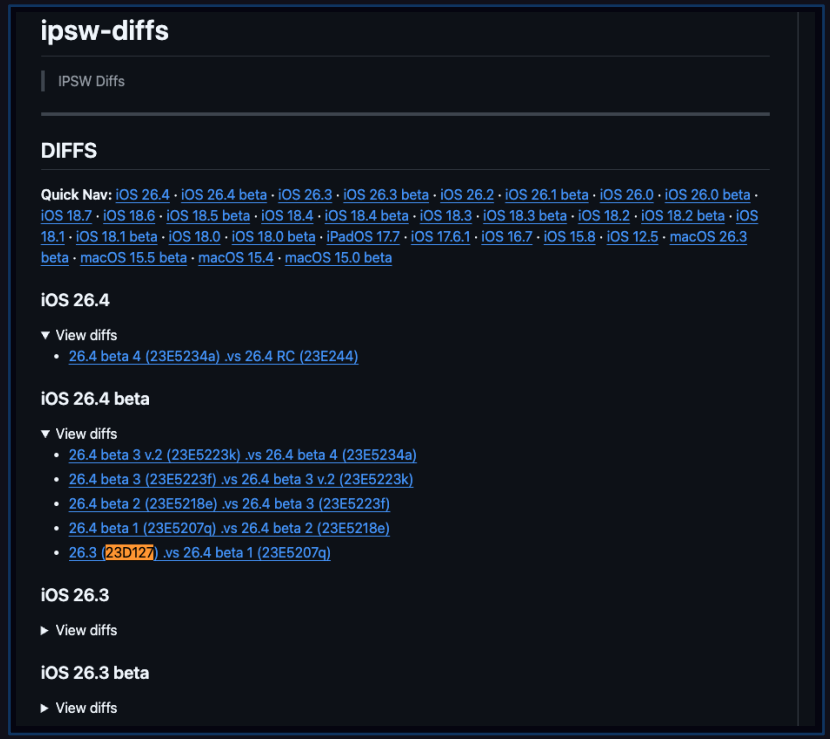
```
→ ipsw-ios ipsw diff iPhone17,2_26.2.1_23C71_Restore.ipsw iPhone17,2_26.3_23D127_Restore.ipsw
  • Diffing KERNELCACHES
  • Diffing DYLD_SHARED_CACHES
  • Mounting 'Old' SystemOS DMG
```

```
$ ipsw-ios ipsw diff iPhone17,2_26.2.1_23C71_Restore.ipsw iPhone17,2_26.3_23D127_Restore.ipsw
```

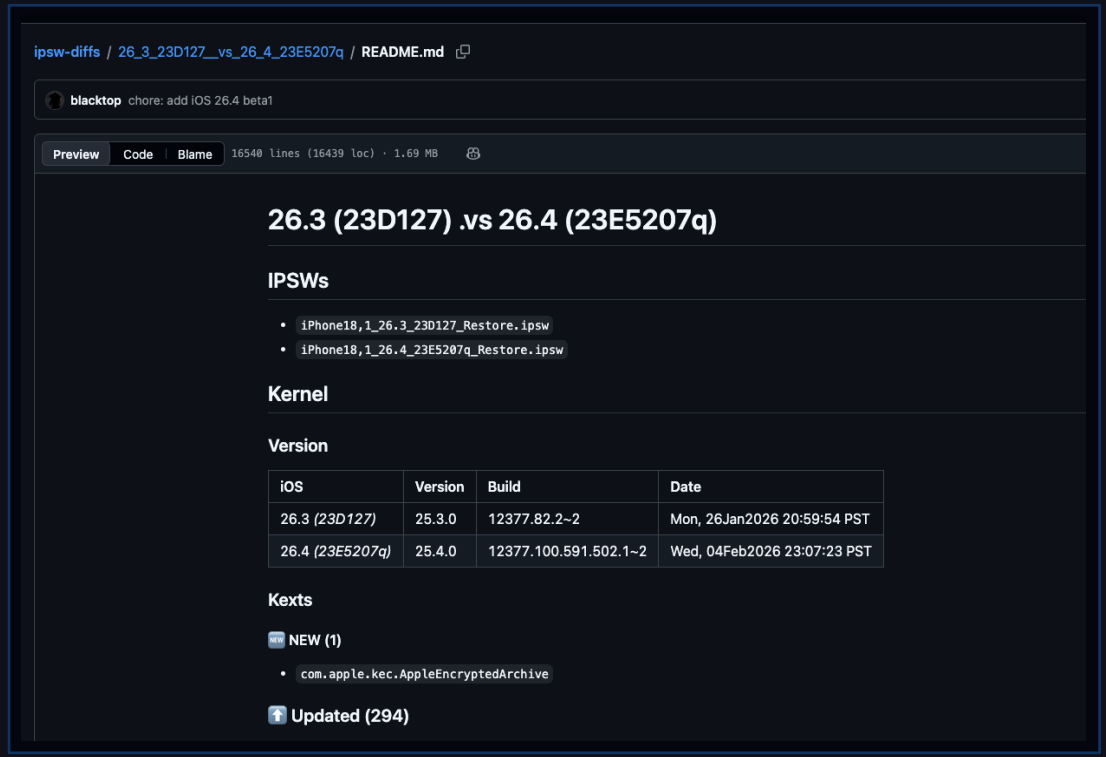
IPSW Diffing Públicos — blacktop/ipsw-diffs



IPSW Diffing Públicos — ipsw-diffs.com



IPSW Diffing Públicos — 26.3 vs 26.4



ipsw-diffs / 26_3_23D127__vs_26_4_23E5207q / README.md

blacktop chore: add iOS 26.4 beta1

Preview | Code | Blame 16540 lines (16439 loc) · 1.69 MB

26.3 (23D127) .vs 26.4 (23E5207q)

IPSWs

- `iPhone18,1_26.3_23D127_Restore.ipsw`
- `iPhone18,1_26.4_23E5207q_Restore.ipsw`

Kernel

Version

iOS	Version	Build	Date
26.3 *(23D127)*	25.3.0	12377.82.2~2	Mon, 26Jan2026 20:59:54 PST
26.4 *(23E5207q)*	25.4.0	12377.100.591.502.1~2	Wed, 04Feb2026 23:07:23 PST

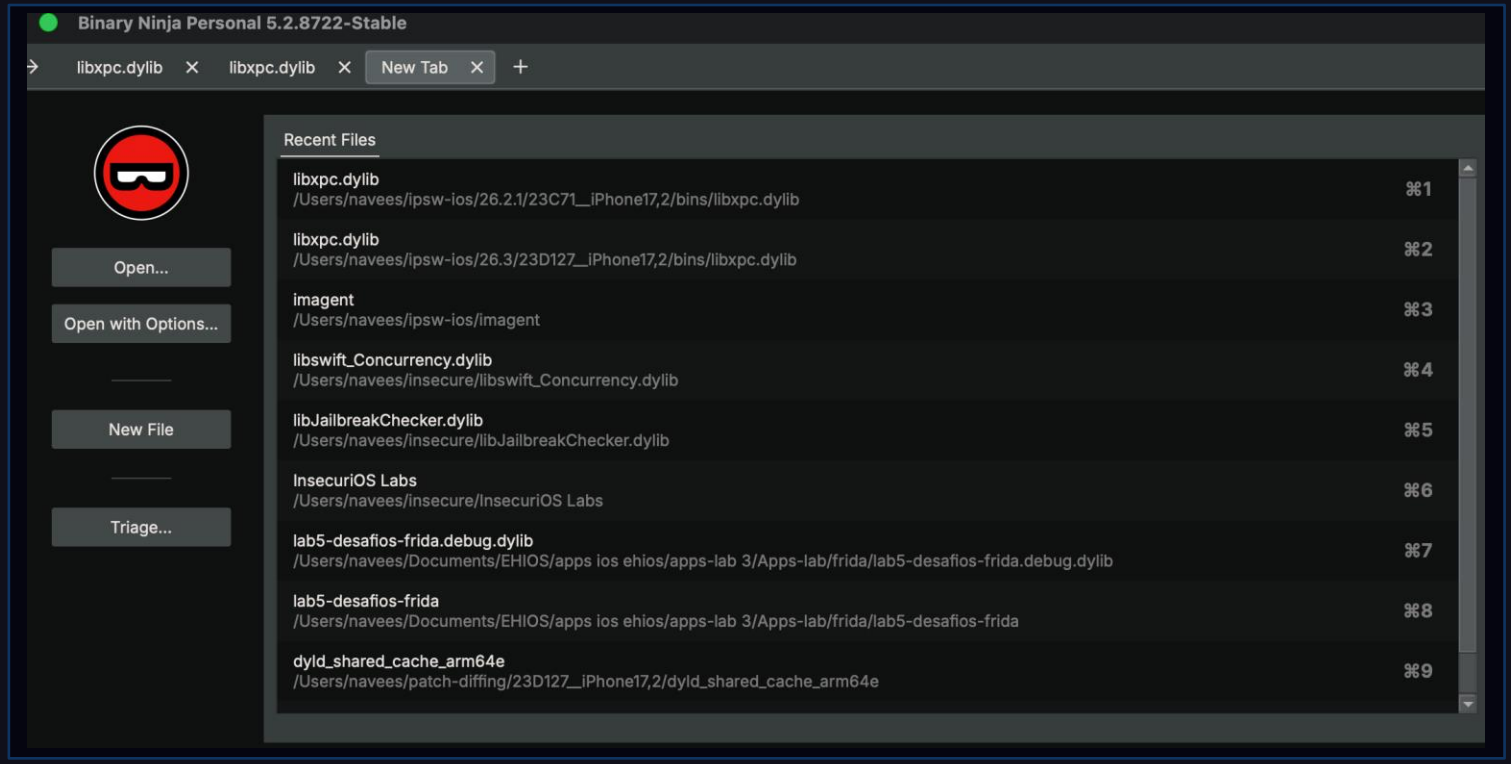
Kexts

🆕 NEW (1)

- `com.apple.kec.AppleEncryptedArchive`

⬆️ Updated (294)

Binary Ninja — Análise de libxpc.dylib



Ghidriff — Ghidra Binary Diff Engine



PYTEST PASSING PYPI DOWNLOADS 1.2K/MONTH STARS 744

Ghidriff - Ghidra Binary Diffing Engine

`ghidriff` provides a command-line binary diffing capability with a fresh take on diffing workflow and results.

It leverages the power of Ghidra's ProgramAPI and FlatProgramAPI to find the *added*, *deleted*, and *modified* functions of two arbitrary binaries. It is written in Python3 using `pyghidra` to orchestrate Ghidra and `jpype` as the Python to Java interface to Ghidra.

Its primary use case is patch diffing. Its ability to perform a patch diff with a single command makes it ideal for automated analysis. The diffing results are stored in JSON and rendered in markdown (optionally side-by-side HTML). The markdown output promotes "social" diffing, as results are easy to publish in a gist or include in your next writeup or blog post.

Diaphora / BinDiff

Diaphora

δiaphora



Diaphora (διαφορά, Greek for 'difference') version 3.1.2 is the most advanced program diffing tool (working as an IDA plugin) available as of today (2024). It was released first during SyScan 2015 and has been actively maintained ever since: Diaphora has been ported to every single minor version of IDA since 6.8 to 8.4.

Diaphora supports versions of IDA >= 7.4 because the code only runs in Python 3.X (Python 3.11 was the last version being tested).

Plugin IDA Pro — IDA >= 7.4
Python 3.X (3.11 tested)

BinDiff

BinDiff

This repository contains the BinDiff source code. BinDiff is an open-source comparison tool for binary files to quickly find differences and similarities in disassembled code.

Table of Contents

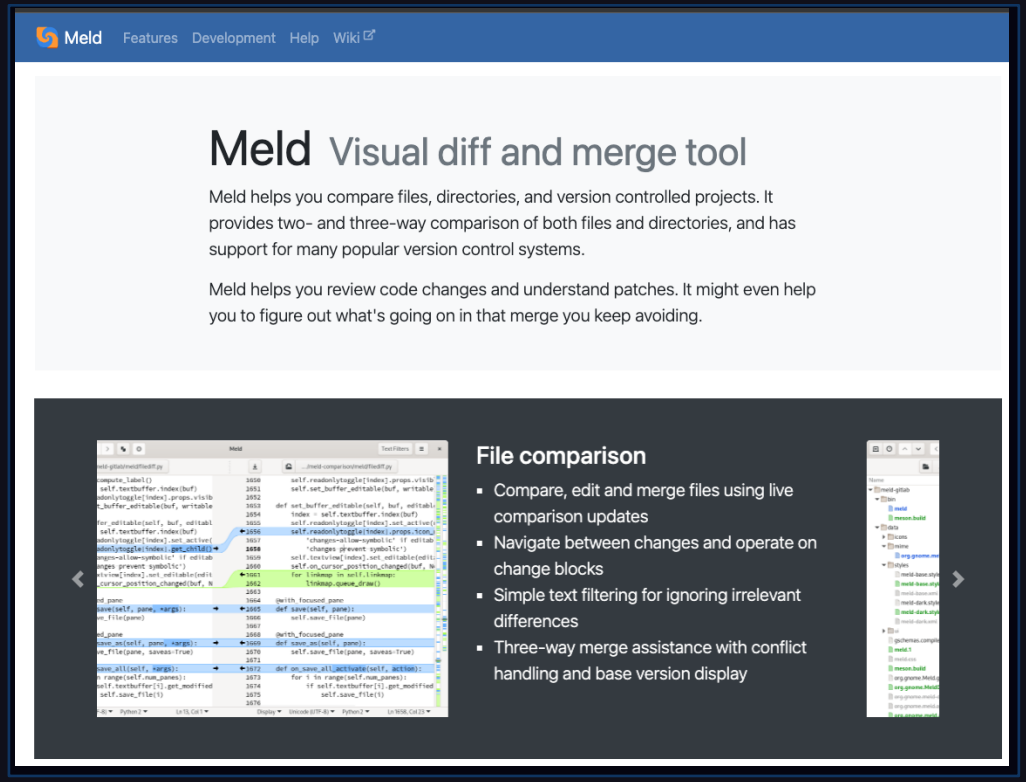
- About BinDiff
- Quickstart
- Documentation
- Codemap
- Building from Source
- License
- Getting Involved

About BinDiff

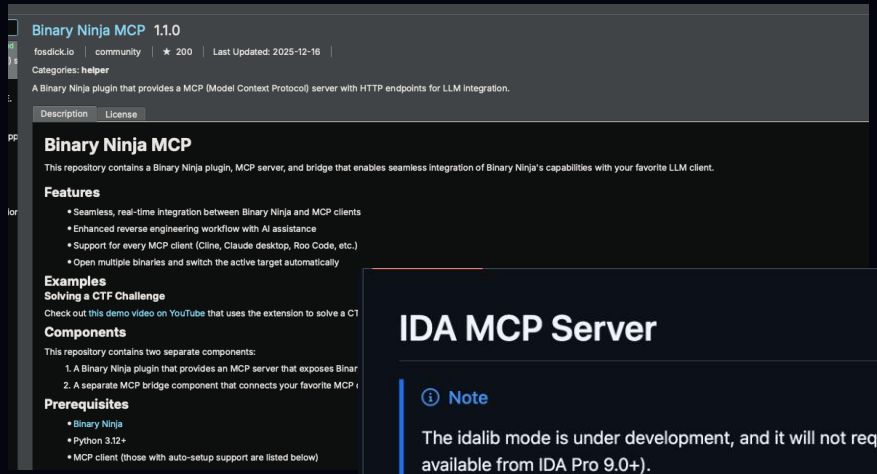
BinDiff is an open-source comparison tool for binary files, that assists vulnerability researchers and engineers to quickly find differences and similarities in disassembled code.

With BinDiff, researchers can identify and isolate fixes for vulnerabilities in vendor-supplied patches. It can also be used to port symbols and comments between disassemblies of multiple versions of the same binary. This makes tracking changes over time easier and allows organizations to retain analysis results and enables knowledge transfer among binary analysts.

Meld — Visual Diff and Merge Tool



MCP Binary Ninja/Ghidra/IDA





IDA MCP Server

> **Note**
>
> The idalib mode is under development, and it will not require installing the IDA plugin or running IDA (idalib is available from IDA Pro 9.0+).

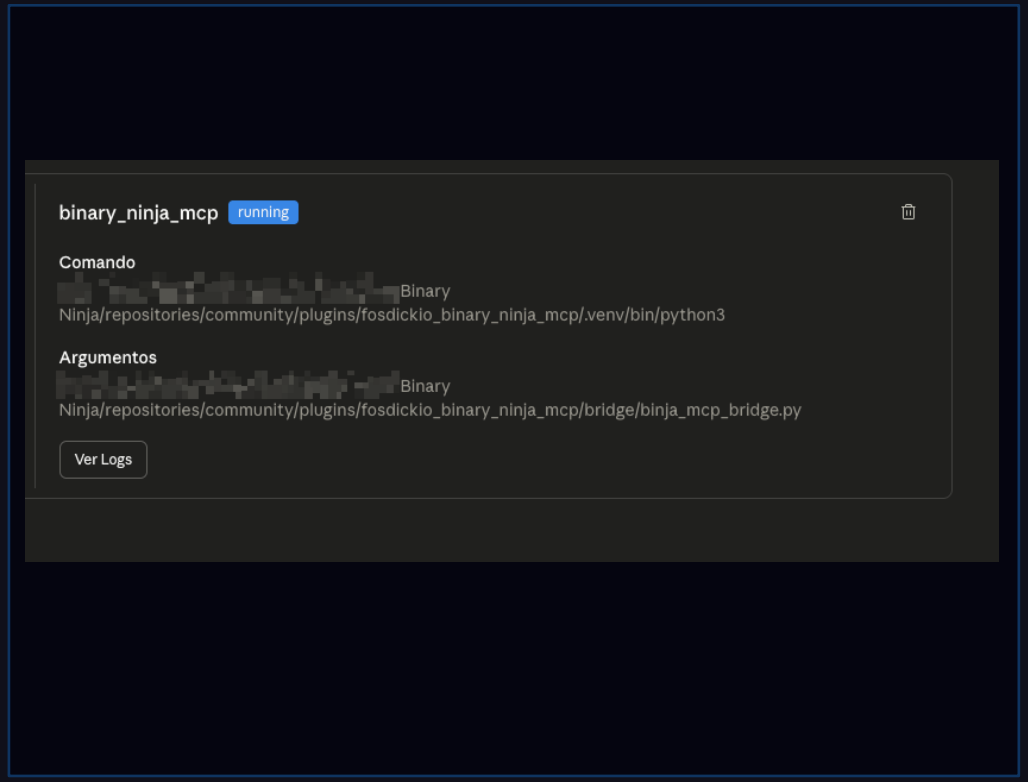
Overview

A Model Context Protocol server for IDA interaction and automation. This server provides tools to read IDA database via Large Language Models.

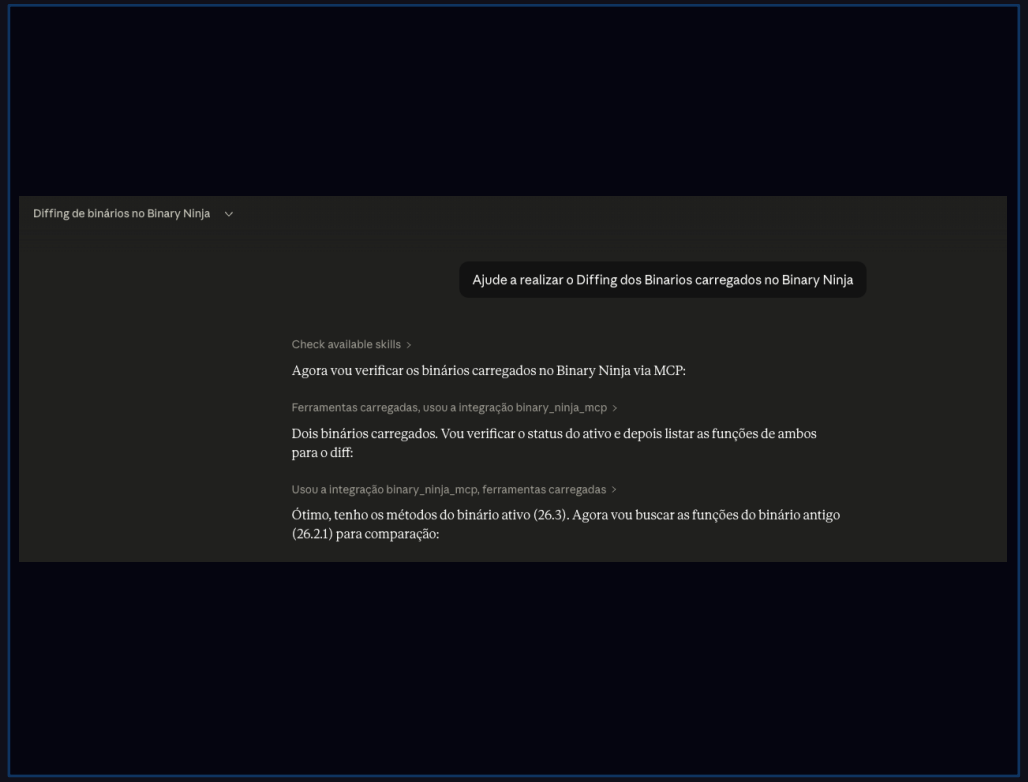
Please note that mcp-server-ida is currently in early development. The functionality and available tools are subject to change and expansion as we continue to develop and improve the server.

to autonomously reverse engineer applications. It
ents.

MCP Binary Ninja



MCP Binary Ninja



05



CVE-2026-20667

libxpc · Sandbox Escape
Anonymous Researcher

Identificando CVE-2026-20667

libxpc

Available for: iPhone 11 and later, iPad Pro 12.9-inch 3rd generation and later, iPad Pro 11-inch 1st generation and later, iPad Air 3rd generation and later, iPad 8th generation and later, and iPad mini 5th generation and later

Impact: An app may be able to break out of its sandbox

Description: A logic issue was addressed with improved checks.

CVE-2026-20667: an anonymous researcher

Análise CVE-2026-20667 — ipsw diff

```
ipsw ipsw diff iPhone17,2_26.2.1_23C71_Restore.ipsw iPhone17,2_26.3_23D127_Restore.ipsw --output diff/ --markdown
  • Diffing KERNELCACHES
  • Diffing DYLD_SHARED_CACHES
  • Mounting 'Old' SystemOS DMG
    • Mounting 043-88290-079.dmg
  • Mounting 'New' SystemOS DMG
    • Mounting 043-48094-150.dmg
  • Diffing MachOs
  • Scanning FileSystem
  • Scanning SystemOS
  • Scanning AppOS
  • Scanning ExclaveOS
  • Scanning FileSystem
  • Scanning SystemOS
  • Scanning AppOS
  • Scanning ExclaveOS
    • Unmounting 'Old' SystemOS DMG
    • Unmounting 'New' SystemOS DMG
    • Deleting 'Old' SystemOS DMG
    • Deleting 'New' SystemOS DMG
  • Creating diff file Markdown README: diff/26_2_1_23C71__vs_26_3_23D127/README.md
```

Análise CVE-2026-20667 — Extraindo dyld_shared_cache

```
→ ipsw-ios ipsw extract --dyld iPhone17,2_26.2.1_23C71_Restore.ipsw -o ./26.2.1/

  • Extracting dyld_shared_cache
    • Mounting DMG /var/folders/7f/2ftfyj7n485bm78007g85mq40000gn/T/ipsw_extract_dyld885701013/043-88290-079.dmg
    • Created 26.2.1/23C71__iPhone17,2/dyld_shared_cache_arm64e
    • Created 26.2.1/23C71__iPhone17,2/dyld_shared_cache_arm64e.01
    • Created 26.2.1/23C71__iPhone17,2/dyld_shared_cache_arm64e.02
    • Created 26.2.1/23C71__iPhone17,2/dyld_shared_cache_arm64e.03
```

```
→ ipsw-ios ipsw extract --dyld iPhone17,2_26.3_23D127_Restore.ipsw  -o ./26.3/

  • Extracting dyld_shared_cache
    • Mounting DMG /var/folders/7f/2ftfyj7n485bm78007g85mq40000gn/T/ipsw_extract_dyld3632640259/043-48094-150.dmg
    • Created 26.3/23D127__iPhone17,2/dyld_shared_cache_arm64e
    • Created 26.3/23D127__iPhone17,2/dyld_shared_cache_arm64e.01
    • Created 26.3/23D127__iPhone17,2/dyld_shared_cache_arm64e.02
    • Created 26.3/23D127__iPhone17,2/dyld_shared_cache_arm64e.03
    • Created 26.3/23D127__iPhone17,2/dyld_shared_cache_arm64e.04
```

Análise CVE-2026-20667 — Binário Vulnerável (26.2.1)

```
1973c95e0            if ((__xpc_has_prefix() & 1) == 0)
1973c95ec                *(arg1 + 0x130) = "linked executable"
1973c95f4                __builtin_memset(dest: &var_68, ch: 0, count: 0x20)
1973c95f4
⚠ 1973c9608              if ((0x19a988800(&__macho_header, &var_58) & 1) == 0)
1973c960c                    var_58 = 0
1973c960c                    int64_t var_50_1 = 0
1973c960c
1973c9618                int64_t x3_2
1973c9618
⚠ 1973c9618              if ((0x19a988830(&var_68) & 1) == 0)
1973c9750                    x3_2 = 0
1973c9754                    var_68 = 0
1973c9754                    int64_t var_60_3 = 0
1973c9618                else
1973c961c                    var_70 = 0
⚠ 1973c9628                  x3_2 = 0x19a988820(&var_70)
1973c9628
1973c9764                int64_t var_80_3 = *(arg1 + 0x30)
1973c9764                char const* const var_78_3 = "linked executable"
⚠ 1973c9784              0x19a988910(&__macho_header, &var_58, &var_68, x3_2, 0,
1973c9784                    "com.apple.xpc.bundle", "bundle %s tainted with reason "%s"")
1973c9784
1973c978c            __xpc_bundle_normalize_cryptex_path(arg1 + 0xe8)
1973c9790            result = 0
1973c958c        else
1973c9590            result = result_1
1973c9590
1973c9598            if (result_1.d != 2)
1973c9638                *(arg1 + 0x130) = "failed to resolve executable"
1973c9640                __builtin_memset(dest: &var_68, ch: 0, count: 0x20)
1973c9640
⚠ 1973c9651              if ((0x19a988800(&__macho_header, &var_58) & 1) == 0)
```

Análise CVE-2026-20667 — Binário Corrigido (26.3)

```
19752a758                    var_58 = 0
19752a758                    int64_t var_50_3 = 0
19752a758
19752a764                int64_t x3_4
19752a764
19752a764                if ((0x19aab0880(&var_68) & 1) == 0)
19752a7f8                    x3_4 = 0
19752a7fc                    var_68 = 0
19752a7fc                    int64_t var_60_3 = 0
19752a764                else
19752a768                    var_70 = 0
19752a774                    x3_4 = 0x19aab0870(&var_70)
19752a774
19752a80c                int64_t var_80_3 = *(arg1 + 0x30)
19752a80c                char const* const var_78_3 = "linked executable"
19752a82c                0x19aab0960(&__macho_header, &var_58, &var_68, x3_4, 0,
19752a82c                    "com.apple.xpc.bundle", "bundle %s tainted with reason "%s"")
19752a830                result = 0xa4
19752a678            else
19752a680                __xpc_bundle_normalize_cryptex_path(arg1 + 0xe8)
19752a684                result = 0
19752a624        else
19752a628            result = result_1
19752a628
19752a630            if (result_1.d != 2)
19752a694                *(arg1 + 0x130) = "failed to resolve executable"
19752a69c                __builtin_memset(dest: &var_68, ch: 0, count: 0x20)
19752a69c
19752a6b0                if ((0x19aab0850(&__macho_header, &var_58) & 1) == 0)
19752a6b4                    var_58 = 0
19752a6b4                    int64_t var_50_1 = 0
19752a6b4
```

Análise CVE-2026-20667 — Comparação Visual (Meld)



```
        if (!(__xpc_has_prefix() & 1))
        {
            *(arg1 + 0x130) = "linked executable";
            __builtin_memset(&var_68, 0, 0x20);

!             if (!(0x19a988800(&__macho_header, &var_58) & 1))
            {
                var_58 = 0;
                int64_t var_50_1 = 0;
            }

            int64_t x3_2;

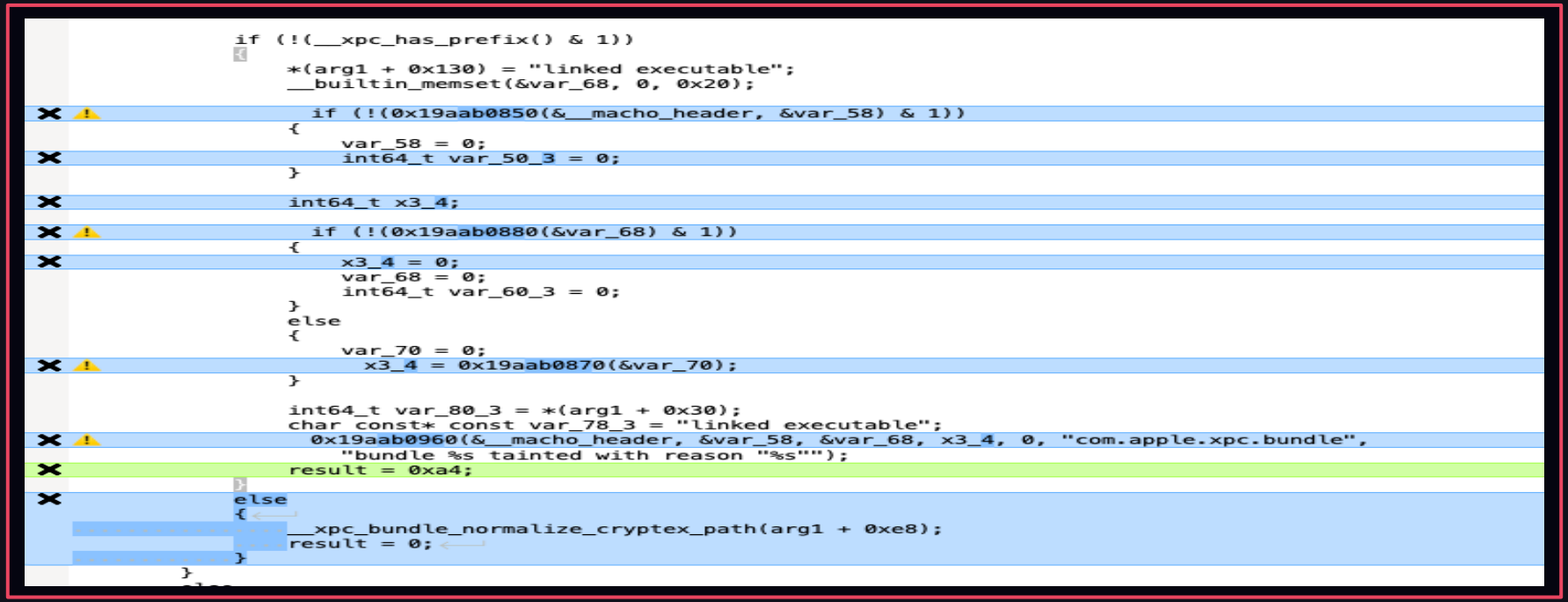
!             if (!(0x19a988830(&var_68) & 1))
            {
                x3_2 = 0;
                var_68 = 0;
                int64_t var_60_3 = 0;
            }
            else
            {
                var_70 = 0;
!                 x3_2 = 0x19a988820(&var_70);
            }

            int64_t var_80_3 = *(arg1 + 0x30);
            char const* const var_78_3 = "linked executable";
!           0x19a988910(&__macho_header, &var_58, &var_68, x3_2, 0, "com.apple.xpc.bundle",
                "bundle %s tainted with reason "%s"");
        }

        __xpc_bundle_normalize_cryptex_path(arg1 + 0xe8);
        result = 0;
    }
    else
    {
        result = result_1;

        if (result_1 != 2)
        {
            *(arg1 + 0x130) = "failed to resolve executable";
            __builtin_memset(&var_68, 0, 0x20);
```

CVE-2026-20667 — Root Cause (Patch de 4 bytes)



```
            if (!(__xpc_has_prefix() & 1))
            {
                *(arg1 + 0x130) = "linked executable";
                __builtin_memset(&var_68, 0, 0x20);

                  if (!(0x19aab0850(&__macho_header, &var_58) & 1))
                {
                    var_58 = 0;
                    int64_t var_50_3 = 0;
                }

                int64_t x3_4;

                  if (!(0x19aab0880(&var_68) & 1))
                {
                    x3_4 = 0;
                    var_68 = 0;
                    int64_t var_60_3 = 0;
                }
                else
                {
                    var_70 = 0;
                      x3_4 = 0x19aab0870(&var_70);
                }

                int64_t var_80_3 = *(arg1 + 0x30);
                char const* const var_78_3 = "linked executable";
                  0x19aab0960(&__macho_header, &var_58, &var_68, x3_4, 0, "com.apple.xpc.bundle",
                    "bundle %s tainted with reason "%s"");
                result = 0xa4;
            }
            else
            {
                __xpc_bundle_normalize_cryptex_path(arg1 + 0xe8);
                result = 0;
            }
        }
```

result = 0x4 → Sandbox escape via __xpc_bundle_resolve_executable · Fix: single 4-byte patch

Resumo



Patch diffing é um dos métodos mais poderosos para entender vulnerabilidades no iOS — todo patch da Apple revela a causa raiz quando comparado.



Binários não mentem. Um patch é uma divulgação pública de uma vulnerabilidade.



No iOS, isso envolve trabalhar com arquivos IPSW, kernelcache e dyld_shared_cache.



Essencial para análise de causa raiz de CVEs e para confirmar a eficácia dos patches de segurança da Apple.



Faca de dois gumes: ajuda pesquisadores a fortalecer a segurança, mas também fornece a atacantes insights sobre vulnerabilidades recém-corrigidas.

Obrigado!

Perguntas & Discussão

Twitter / X: @naveeeees

Linkedin: linkedin.com/in/brunonaves/



⚠ CVE-2026-20667 — Informações divulgadas pela Apple.
Pesquisa realizada de forma ética e responsável.

$ ./talk --end → EOF